

Hybrid Biometric Locomotion

Ground-Walk + Sky-Flight

peiwenzhang

M1 HCI

Université Paris-Saclay

Project Overview

- **Core Design:**

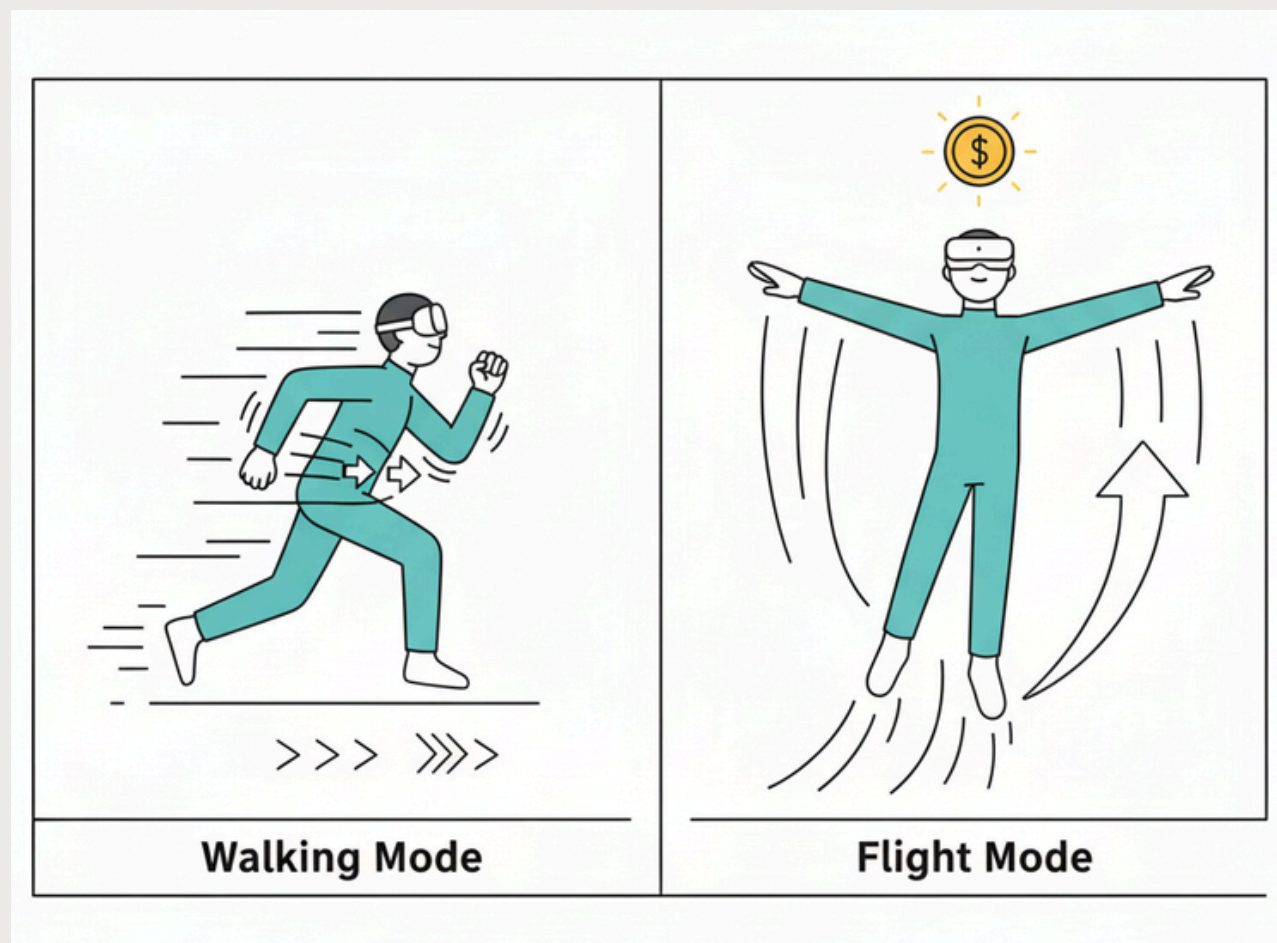
Seamlessly combining walking on the ground and flying in the sky.

Walking Mode:

Move like natural walking or running. You move by swinging your arms.

Flight Mode:

"Fly like a bird." Look where you want to go (Gaze-directed) and flap your arms to fly in 3D space.



Core Goal:

Increase immersion and reduce motion sickness using physical feedback.

Interaction Mechanism: The "Trigger Clutch"

Concept:

Using the Index Trigger as a "Clutch" for movement.

- Hold to Move: Movement logic only activates when the trigger is held.
- Release to Stop: Instant "Emergency Brake" when released. This prevents drifting and reduces motion sickness.



Seamless Transition:

- Walk: Look forward + Swing arms (Y-axis is naturally low).
- Fly: Look up + Swing arms (Unlocks the vertical dimension).

```
// The movement only works when the trigger is held
bool isTriggerHeld = OVRInput.Get(OVRInput.Button.PrimaryIndexTrigger) ||
    OVRInput.Get(OVRInput.Button.SecondaryIndexTrigger);

if (isTriggerHeld && swingPower > 0.08f) {
    // Perform movement...
} else {
    // Instant braking or smooth damping
    moveVelocity = Vector3.Lerp(moveVelocity, Vector3.zero, Time.deltaTime * damping);
}
```

Technical Implementation

- Input: Calculates the combined velocity of both controllers (swingPower).
- Walk Mode: Direction is based on the HMD's forward vector. When looking straight, movement is horizontal.
- Fly Mode: Direction follows the gaze. Looking up adds vertical thrust for 3D navigation.
- Speed Control: A quadratic curve ($P^{2.5}$) filters out small hand shakes and allows for natural acceleration (from slow walk to fast run/flight).

Quadratic Curve

$$P = V^2$$

```
// (Sum of hand speeds)
float swingPower = OVRInput.GetLocalControllerVelocity(OVRInput.Controller.LTouch).magnitude +
    OVRInput.GetLocalControllerVelocity(OVRInput.Controller.RTouch).magnitude;

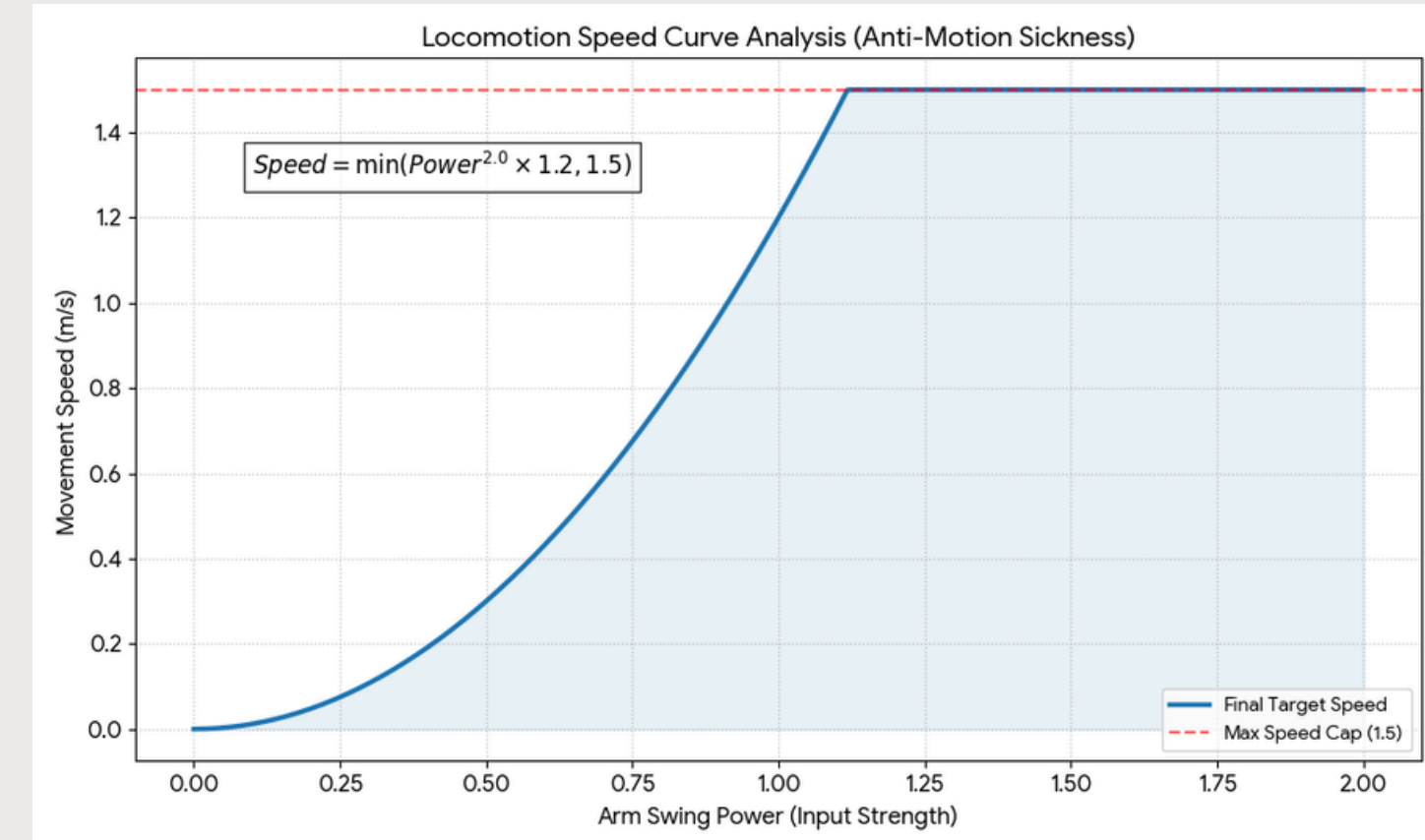
// (Speed Curve)
// The faster you swing, the faster you go, simulating the burst from walking to running.
float modulatedPower = Mathf.Pow(swingPower, 2.5f);
float targetSpeed = Mathf.Min(modulatedPower * sensitivity, maxSpeed);
```

```
// 1. Get the direction (Where the player is looking)
Vector3 lookDirection = hmd.transform.forward;

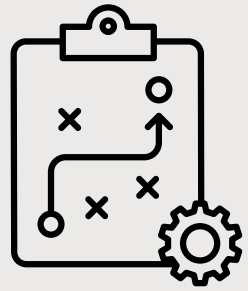
// 2. Apply speed to that direction (3D Movement)
// Look down to fly down; Look up to fly up; Look forward to walk
Vector3 targetVelocity = lookDirection * targetSpeed;

// 3. Make the movement smooth
moveVelocity = Vector3.Lerp(moveVelocity, targetVelocity, Time.deltaTime * 5f);
controller.Move(moveVelocity * Time.deltaTime);
```

4



Challenges & Solutions



1. Script Conflict

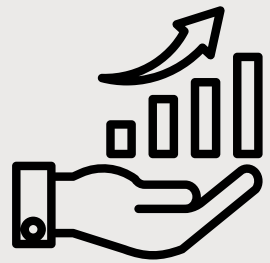
Problem: Default Unity scripts and my new code fight each other.

Solution: Automatically find and disable old scripts when the game starts.

```
// Example of how I find and connect to other scripts automatically
void Start() {
    // Automatically find the Parkour counter
    if (parkourCounter == null)
        parkourCounter = Object.FindFirstObjectByType<ParkourCounter>();

    // Link this movement script to the game manager
    if (parkourCounter != null)
        parkourCounter.locomotionTech = this;
}
```

Challenges & Solutions



2. Hitting "Walls"

Problem: The banners (finish lines) were like solid walls. The player would stop or bounce back.

Solution: Use a script to change Banners into "Triggers" (Sensors) so players can pass through them.

```
// Change all banners from "Walls" to "Sensors"
void FixBannerPhysics() {
    // 1. Find all objects with the "banner" tag
    GameObject[] banners = GameObject.FindGameObjectsWithTag("banner");

    foreach (GameObject b in banners) {
        Collider c = b.GetComponent<Collider>();
        // 2. If it is a solid wall, change it to a Trigger
        if (c != null && !c.isTrigger) {
            c.isTrigger = true;
            Debug.Log("Fixed: " + b.name + " is now a Trigger");
        }
    }
}
```

Evaluation Setup

1



2



3



Participants: 3 (myself + 2 male).

Test task: Complete the entire Parkour course and collect as many coins as possible.

Participant	Loco 1 Time	Loco 2 Time	Loco 3 Time	Total Time	Coins Collected (Total 69)
Me	48.9s	52.3s	30.9s	132.1s	68 / 69
Participant A	55.2s	58.6s	35.4s	149.2s	65 / 69
Participant B	42.5s	49.8s	28.7s	121.0s	62 / 69

Subjective Measures (1-10 scale)

Evaluation Dimension	Me	Participant A	Participant B	Average Score
Sickness (1 low - 10 high)	2	4	3	3.0
Workload (1 easy - 10 difficult)	3	5	4	4.0
Presence (1 low - 10 high)	8	7	8	7.7
Enjoyment (1 low - 10 high)	9	8	9	8.7

Main Findings

High Comfort (Low Sickness: 3.0/10)

- Movement is smooth. No dizziness.
- The "Dynamic Curve" works perfectly.

Speed vs. Accuracy

Fast speed = More missed coins.

Goal: Improve control for high-speed flight.

Great Experience (Enjoyment: 8.7/10)

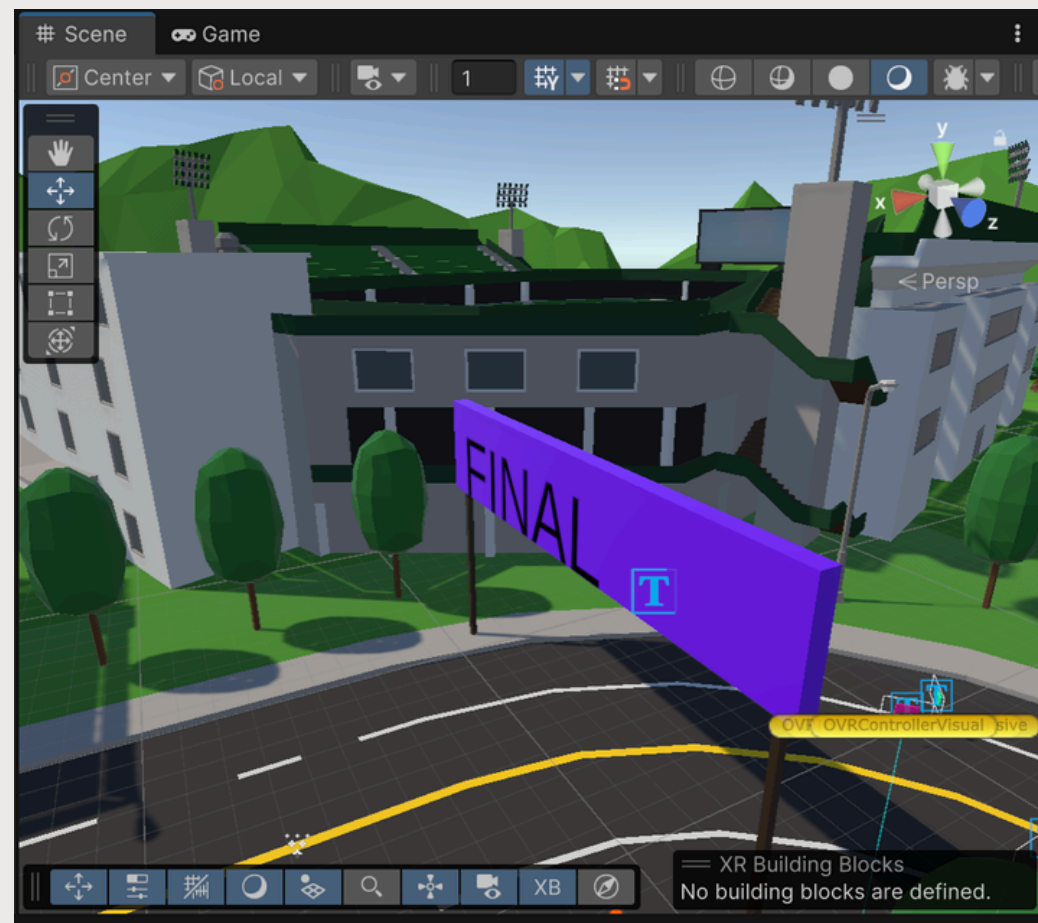
It feels real, like "flying like a bird."

High presence (7.7/10).



Bonus: Paris-Saclay Flavor

As a student at Université Paris-Saclay, I incorporated the school's color (purple) into the project:



The Final Banner was changed to purple.



The last three coins were given a glowing purple material effect.

Demo



Part 1:walking mode

Part 2:running mode

Part 3:flying mode

The
End



Thank you

peiwenzhang